QA Testing Document

Everest View ERP School Management System

Document Type: Quality Assurance, Test Plan, Test Cases, and Test Strategy

Project: ERP School / Student ERP

Framework: Next.js, React, MongoDB, NextAuth.js, Cypress

Prepared On: June 21, 2026

Table of Contents

1. QA Executive Summary

This QA Testing Document defines the quality assurance process for the Everest View ERP School Management System. The project is a role-based school ERP application with public pages, owner administration, teacher portal, student portal, protected API routes, authentication, fee management, academic records, notices, assignments, attendance, examinations, and reports.

The QA goal is to verify that the ERP works correctly for all major user roles and that the system remains secure, usable, responsive, and reliable. The project already includes Cypress end-to-end tests organized across public, owner, teacher, student, shared, and API test folders. This document formalizes the testing approach, expected coverage, test cases, regression strategy, and sign-off checklist.

Current Automated QA Structure

- 34 Cypress spec files are present under cypress/e2e.
- Coverage areas include public pages, owner pages, teacher pages, student pages, shared sidebar navigation, and API routes.
- Cypress is configured with baseUrl http://localhost:3000, screenshot capture on failure, and role session helpers.
- Custom commands support owner, teacher, and student login/session setup.

2. QA Objectives

The QA process validates that the ERP meets functional requirements and provides a stable user experience. Testing must cover both normal workflows and failure conditions, especially because the application stores academic, attendance, and financial data.

Primary QA Objectives

- Verify that public pages load correctly and navigation works as expected.
- Verify that login, logout, session handling, and role-based redirects work correctly.
- Verify owner workflows for students, teachers, classes, attendance, marks, exams, fees, results, reports, notices, and passout records.
- Verify teacher workflows for dashboard, classes, students, attendance, exams, assignments, marks, and notices.
- Verify student workflows for dashboard, marksheet, assignments, routine, and fees.
- Verify protected API endpoints reject unauthenticated access.
- Verify API responses return expected status codes and response shapes.
- Verify responsive behavior on desktop and mobile layouts.
- Verify that critical actions provide validation, feedback, and confirmation where required.

3. Project Under Test

The application under test is a Next.js school ERP system. It contains public marketing pages and authenticated dashboard sections for three roles: owner, teacher, and student. The backend is implemented through Next.js API routes and MongoDB models.

Project Summary

Application Name	Everest View ERP / ERP School Management System
Framework	Next.js with React
Database	MongoDB with Mongoose models
Authentication	NextAuth.js credentials-based authentication
Payment Flow	eSewa initiate and success callback routes for student fee payment
Date Handling	Bikram Sambat and Gregorian date support for school workflows
Automated Test Tool	Cypress end-to-end testing

Major Functional Areas

- Public landing page, legal pages, login page, and not found page.
- Owner dashboard and complete administration modules.
- Teacher dashboard and classroom management modules.
- Student dashboard and self-service modules.
- REST-style API routes for students, teachers, classes, attendance, marks, fees, exams, assignments, submissions, notices, statistics, and authentication.

4. Test Environment

Testing is designed to run locally against the development server. The Cypress configuration uses a base URL of http://localhost:3000 and browser viewport dimensions of 1280 by 720 for standard desktop testing.

Environment Configuration

Application URL	http://localhost:3000
Cypress Config File	cypress.config.js
Viewport	1280 x 720 by default
Screenshots	Enabled on test failure
Videos	Disabled
Default Timeout	10000 ms
Session Tokens	Generated through createSessionToken task using NEXTAUTH_SECRET

Required Setup Before Test Execution

1. Install dependencies using npm install if node_modules is not present.
2. Create a valid .env file with database connection and NEXTAUTH_SECRET.
3. Start the development server using npm run dev.
4. Run Cypress tests from another terminal using npx cypress run or npx cypress open.
5. Review screenshots and terminal output for failed tests.

> **Important Note**
>
> - The current package.json includes lint, dev, build, and start scripts, but does not define a dedicated test script.
> - Recommended future improvement: add scripts such as test:e2e and test:e2e:open for consistent QA execution.

5. Testing Scope

The QA scope includes functional testing, navigation testing, API testing, role-based access testing, UI testing, and regression testing for all major modules. Testing should focus first on critical school operations and then cover secondary visual and usability issues.

In Scope

- Public website page rendering and navigation.
- Login page behavior, valid and invalid credential handling, and session redirects.
- Owner administration screens and critical workflows.
- Teacher portal screens and assigned-class workflows.
- Student portal screens and self-service workflows.
- Protected API endpoint access rules and response format checks.
- Sidebar and dashboard navigation.
- Form controls, modals, tabs, filters, buttons, and empty states.

Out of Scope for Current Automated Suite

- Load testing with a large production-sized database.
- Native mobile application testing.
- Real payment gateway settlement verification beyond callback handling.
- Accessibility audit using automated WCAG tooling.
- Cross-browser matrix execution across all major browsers unless Cypress is configured for it.

6. Testing Types

Functional Testing

Functional testing verifies that each ERP feature works according to expected behavior. This includes page loading, form submission, modal display, filters, tabs, CRUD flows, dashboard cards, payment sections, and report displays.

End-to-End Testing

End-to-end testing verifies real user flows across pages. Cypress tests are used for user-facing workflows such as login, owner navigation, teacher page access, student dashboard viewing, and API checks.

API Testing

API testing verifies status codes, authentication rules, and response shapes for backend routes. The existing suite includes unauthenticated checks and owner-authenticated API response validation.

Regression Testing

Regression testing should be executed after every meaningful change to authentication, data models, API routes, UI pages, or shared components to ensure existing modules still work.

Security Testing

Security testing focuses on unauthenticated access rejection, role-based route protection, safe session handling, and protection of sensitive student, teacher, and fee data.

7. Cypress Test Architecture

The Cypress test suite is organized by application area. This makes it easy to run or review tests for a specific role or module. Custom commands reduce repetition for login and session creation.

Cypress Folder Coverage

cypress/e2e/public	Landing page, login page, legal pages, layout, and 404 page tests.
cypress/e2e/owner	Owner dashboard and administration module tests.
cypress/e2e/teacher	Teacher dashboard and classroom module tests.
cypress/e2e/student	Student dashboard, assignments, marksheet, routine, and fees tests.
cypress/e2e/shared	Shared sidebar navigation tests for all roles.
cypress/e2e/api	Authentication, public endpoint, protected endpoint, and owner API tests.
cypress/support/commands.js	Reusable login/session helpers for owner, teacher, and student roles.

Session Strategy

- Owner login uses the login page and expected owner credentials.
- Teacher and student tests use mocked session/API responses and generated NextAuth session tokens.
- The createSessionToken task encodes role-specific user data using NEXTAUTH_SECRET.
- API tests use cy.request to check endpoint status and response structure.

8. Public Website Test Cases

Public website testing verifies that unauthenticated users can access the landing page, legal information, login page, and custom not found page. These tests protect the first impression and public navigation of the school ERP.

Public Test Coverage

- Home page loads and displays the main school/ERP sections.
- Header, footer, public layout, and navigation links render correctly.
- Login page displays identifier and password fields.
- Legal pages for privacy, terms, and cookies are accessible.
- Invalid routes display the custom 404 page.

Recommended Manual Checks

- Verify smooth-scroll links on the landing page.
- Verify public pages on small mobile screens.
- Verify contact form success and failure states with valid and invalid input.
- Verify newsletter subscribe and unsubscribe flows.

9. Authentication and Authorization Test Cases

Authentication and authorization are critical because the ERP contains private school data. Testing must confirm valid login, invalid login, route protection, session persistence, and role separation.

Existing Auth API Checks

- Rejects login with missing credentials.
- Allows login with owner credentials.
- Rejects login with wrong password.

Role-Based Access Test Cases

1. Unauthenticated user visits /owner/dashboard and should be redirected to /login.
2. Unauthenticated user visits /teacher/dashboard and should be redirected to /login.
3. Unauthenticated user visits /student/dashboard and should be redirected to /login.
4. Owner user should access owner dashboard and owner module pages.
5. Teacher user should access teacher dashboard and teacher module pages.
6. Student user should access student dashboard and student module pages.
7. Teacher and student users should not access owner-only pages.
8. Logout should clear the authenticated session and return user to public flow.

10. Owner Module Test Cases

The owner role has the broadest access and must receive the highest QA attention. Owner tests should verify all management modules, data display, controls, forms, and confirmation behavior.

Owner Dashboard

- Dashboard loads with KPI cards, greeting, quick actions, and system summary.
- Quick action cards navigate to students, teachers, fees, exams, attendance, and reports.
- System health and statistics sections handle loading and empty states.

Student Management

- Students page displays header, search, grade filter, Add Student, and Promote All controls.
- Add Student button opens the student modal.
- Student records show name, ID, grade, section, email, attendance, and actions.
- Promotion flow should require confirmation and handle Grade 12 passout behavior.

Teacher Management

- Teachers page displays list/table and correct columns.
- Add Teacher button opens modal and cancel closes it.
- Search, edit, delete, view, and export controls should behave consistently.

Classes, Attendance, Marks, Exams

- Classes page supports schedule cards and teacher assignment display.
- Attendance page supports register and mark views, class selection, date selection, and P/A status.
- Marks page supports exam type, grade, section, subject selection, marks entry, and grade calculation.
- Exams page supports grade selection, term tabs, and adding subjects.

Fees, Results, Reports, Notices, Passout

- Fees page displays fee structure, overview, and history tabs.
- Results page displays exam and grade selectors plus print option.
- Reports page displays school summary metrics.
- Notices page supports title, content, audience selection, publish, list, and delete behavior.
- Passout page displays graduated students and handles empty state.

11. Teacher Module Test Cases

Teacher QA verifies classroom-focused workflows. Teacher access should remain scoped to assigned classes and should not expose full administrative controls.

Teacher Dashboard

- Dashboard displays welcome banner, assigned schedule section, my students section, and class performance section.
- Dashboard action buttons navigate to teacher classes, students, attendance, and notices.

Teacher Classes and Students

- Classes page displays assigned class schedules or empty state when no classes are assigned.
- Students page displays student information related to teacher assignments.
- Search and class grouping should be clear for classroom use.

Teacher Attendance

- Attendance page loads and shows class selector.
- Teacher can mark attendance for assigned grades only.
- Register view and mark view should be clear and date-aware.

Teacher Exams, Assignments, Marks, Notices

- Exams page displays exam management UI and page description.
- Assignments page displays assignment management UI.
- Marks page displays marks entry UI, exam type selector, and grade selector.
- Notices page displays active notices or empty state.

12. Student Module Test Cases

Student QA validates self-service flows and ensures students only see their own data. The student portal must be easy to use on mobile and desktop.

Student Dashboard

- Student portal loads with dashboard title or welcome content.
- Stats cards display attendance, average score, fee status, and pending assignments.
- Quick links navigate to marksheet, assignments, routine, and fees.
- Quick info sidebar displays student identity and grade details.

Student Marksheet

- Marksheet page displays page header, school letterhead, and progress report title.
- Marks table or empty state appears depending on available data.
- Grade, GPA, total marks, and pass/fail status should calculate correctly.

Student Assignments

- Assignments page lists assigned work or shows empty state.
- Submission flow should accept valid file input and show feedback.
- Submitted status should be visible after submission.

Student Routine and Fees

- Routine page displays exam routine for the student grade.
- Fees page displays bill header, fee summary, due amount, payment section, and history.
- eSewa payment flow should handle success, failure, and error redirects.

13. API Test Cases

API testing confirms backend behavior independent of the UI. The existing suite includes public endpoint checks, unauthenticated protected endpoint checks, authentication checks, and owner-authenticated API response checks.

Public Endpoint Checks

- GET /api/contact should return a controlled response such as 405, 200, or 404 depending on implementation.
- POST /api/subscribe accepts email and returns expected status such as 201, 200, 409, or 400.

Protected Endpoint Checks

- Unauthenticated GET requests to teachers, students, fees, classes, attendance, marks, exams, exam-routines, notices, and owner stats should return 401.

Owner Authenticated API Checks

- GET /api/teachers returns status 200 and an array.
- GET /api/students returns status 200 and an array.
- GET /api/classes returns status 200 and an array.
- GET /api/fees returns status 200 and contains students, payments, and classFees.
- GET /api/exams, /api/exam-routines, /api/attendance, /api/marks, and /api/notices return status 200 with arrays.
- GET /api/owner/stats returns status 200 with students, teachers, revenue, and attendance properties.

14. Data Validation Test Cases

Data validation prevents incorrect school records from entering the database. QA should test both client-side and API-level validation wherever applicable.

Student and Teacher Validation

- Required fields must not accept blank values.
- Student ID and teacher ID should be unique.
- Email fields should reject invalid email formats.
- Password fields should meet the configured minimum requirements.

Academic Validation

- Marks should not be less than zero or greater than full marks.
- Exam pass marks should not exceed full marks.
- Attendance should not be marked for future dates.
- Grade, section, subject, and exam type selections should be required before saving marks.

Fee Validation

- Fee amounts, previous due, scholarship, and paid amount should accept only valid numeric values.
- Partial payment amount should not exceed due amount unless business rules allow adjustment.
- Payment history should preserve transaction records after payment.

15. Security and Access Control Testing

Security testing protects private school information. The ERP must ensure that users only access information and actions allowed by their role.

Security Test Areas

- Protected dashboard pages redirect unauthenticated users to login.
- Protected API endpoints return 401 for unauthenticated requests.
- Students cannot access other students records through UI or API parameters.
- Teachers cannot perform owner-only operations such as managing all fees or deleting all students.
- Session tokens expire according to configured NextAuth behavior.
- Logout removes user access to protected routes.
- Sensitive values such as passwords and secrets are not exposed in responses or UI.

Payment Security Checks

- eSewa success route handles missing or invalid data safely.
- Failed or cancelled payment status should not mark fees as paid.
- Successful payment callbacks should update only the intended student transaction.

16. Responsive and UI Testing

The ERP should work on desktops, tablets, and mobile devices. Responsive testing is important because students and teachers may use mobile phones to access assignments, marks, routines, and notices.

Responsive Checks

- Landing page sections stack correctly on mobile.
- Header and mobile menu work on small screens.
- Dashboard sidebars collapse or become accessible through mobile navigation.
- Tables remain readable through responsive layout, scrolling, or card transformation.
- Forms, modals, and date pickers fit within mobile viewport width.
- Buttons and inputs have enough touch area for mobile use.

UI Consistency Checks

- Page titles and module names are consistent with sidebar navigation.
- Primary actions use consistent styling and placement.
- Success, warning, error, and empty states are visually distinct.
- Print views for marksheets, results, fees, and routines are readable.

17. Regression Testing Strategy

Regression testing should be run whenever code changes may affect existing behavior. In this ERP, shared changes to authentication, layout, API routes, models, or utility functions can impact many modules at once.

Recommended Regression Triggers

- Changes to NextAuth configuration, middleware, or session handling.
- Changes to MongoDB models or API route response shapes.
- Changes to shared components such as Header, Footer, Sidebar, or LayoutWrapper.
- Changes to grading, fee calculation, attendance date conversion, or payment callback logic.
- Changes to route structure or dashboard navigation.

Regression Execution Order

1. Run linting using npm run lint.
2. Build the application using npm run build when feasible.
3. Start the app locally using npm run dev.
4. Run Cypress API tests first to validate backend access rules.
5. Run public and authentication tests.
6. Run owner, teacher, student, and shared navigation tests.
7. Manually verify high-risk workflows such as fees, marks, attendance, and payment callback scenarios.

18. Defect Management Process

Defects found during QA should be recorded clearly so developers can reproduce, prioritize, and fix them. Every bug report should include environment details, user role, route, steps, expected result, actual result, and evidence.

Defect Severity Levels

Critical	System unavailable, login broken for all users, data loss, payment incorrectly marked, or severe security access issue.
High	Core module broken such as attendance save failure, marks not saving, fee calculation incorrect, or owner CRUD failing.
Medium	Feature partially works but has incorrect UI, missing validation, incorrect empty state, or non-critical API issue.
Low	Minor style issue, wording issue, spacing problem, or non-blocking UI inconsistency.

Bug Report Template

- Title: Short description of the issue.
- Environment: Browser, OS, local/staging/production, and app URL.
- Role: Owner, Teacher, Student, or Public user.
- Route: Page or API endpoint where issue occurred.
- Steps to Reproduce: Clear numbered steps.
- Expected Result: What should happen.
- Actual Result: What happened.
- Evidence: Screenshot, Cypress failure screenshot, console error, or network response.
- Severity and Priority: Business impact and fix urgency.

19. QA Execution Checklist

The following checklist should be completed before project delivery or demonstration.

Pre-Test Checklist

- Dependencies installed successfully.
- Environment variables configured, including database URI and NEXTAUTH_SECRET.
- Development server starts without runtime errors.
- Database connection works.
- Test accounts or session mocks are available for owner, teacher, and student roles.

Execution Checklist

- Public page tests executed.
- Authentication tests executed.
- Owner module tests executed.
- Teacher module tests executed.
- Student module tests executed.
- API tests executed.
- Sidebar and navigation tests executed.
- Manual mobile responsive checks completed.
- Critical business workflows manually verified.

Post-Test Checklist

- Failed tests reviewed and categorized.
- Screenshots or logs attached to defect reports.
- Critical and high severity defects fixed and retested.
- Regression suite rerun after fixes.
- QA summary prepared for sign-off.

20. QA Sign-Off Recommendation

The ERP should be considered ready for delivery only when core workflows pass automated and manual QA checks. Because the system handles academic records and financial information, owner, teacher, student, and API tests should be treated as mandatory before final approval.

Recommended Release Criteria

- No open critical defects.
- No open high severity defects in login, role access, attendance, marks, fees, payment, or student records.
- Public, owner, teacher, student, shared navigation, and API Cypress suites pass in the target environment.
- Manual responsive checks pass for the most important screens.
- Documentation PDFs are generated and available for review.

Final QA sign-off should be given after all mandatory tests pass, defects are documented, and stakeholders accept any remaining low-risk limitations.

End of QA Testing Document

This document defines the QA testing process for the Everest View ERP School Management System.

Everest View Secondary Boarding School

ERP School Management System QA Documentation